

Memory

Memory

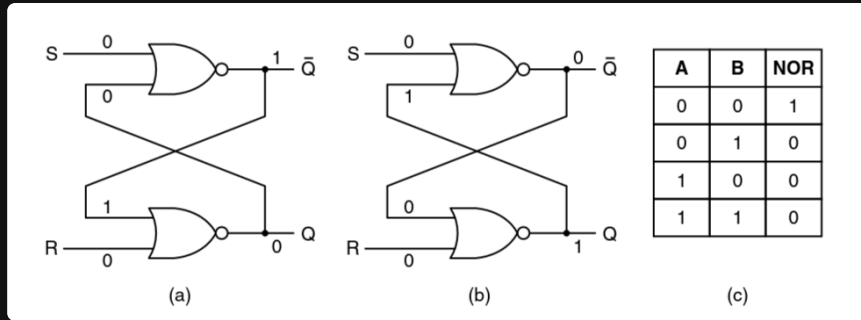
An essential component of every computer is its **memory**. Without memory there could be no computers as we now know them.

Memory is used for storing both *instructions to be executed* and *data*.

We will examine the basic components of a memory system starting at the **gate level**, and see how they combine to build large memories.

Latches

To create a 1-bit memory, we need a circuit that somehow "remembers" previous input values.



Such a circuit can be built from two **NOR** gates. This is called an **SR latch**.

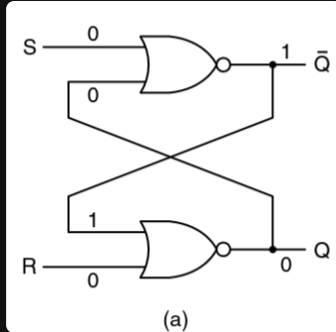
- **S** - for **S**etting the latch
- **R** - for **R**esetting (clearing) it
- **Q** and $\bar{Q}$ - two complementary outputs

Unlike a combinational circuit, the outputs are not uniquely determined by the current inputs alone.

SR Latch

A state where both outputs are 0, or both are 1, is **impossible**

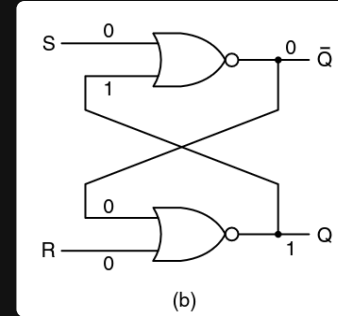
State $Q = 0$



With $S = R = 0$ and $Q = 0$:

- the upper gate sees two 0s,
- so $\bar{Q} = 1$.
- That 1 feeds the lower gate,
- giving $Q = 0$.

State $Q = 1$



With $S = R = 0$ and $Q = 1$:

- the upper gate sees a 0 and 1,
- giving $\bar{Q} = 0$.
- That feeds the lower gate,
- giving $Q = 1$.

SR Latch - Setting and Resetting

Suppose S becomes 1 while $Q = 0$:

- The upper gate now has inputs 1 and 0 $\rightarrow$ forces $\neg Q = 0$
- Both inputs to the lower gate become 0 $\rightarrow$ forces $Q = 1$

So setting S switches the state from 0 to 1.

Setting R while already in state 0 has **no effect**.

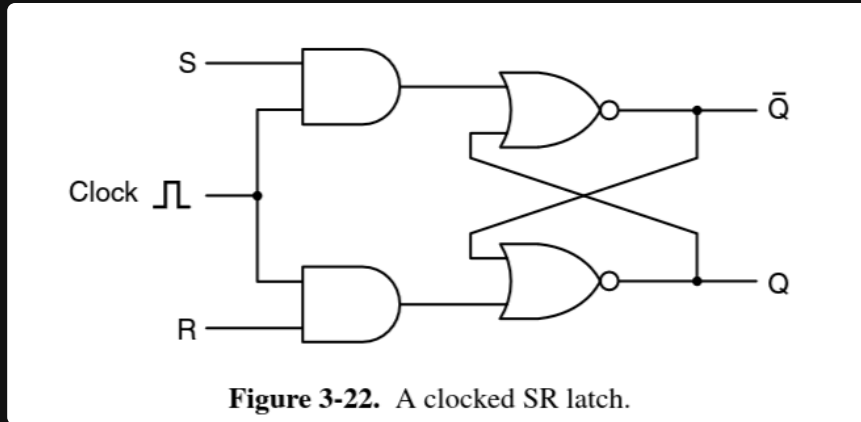
By similar reasoning, setting R to 1 while $Q = 1$ drives the latch to $Q = 0$, while setting S has no effect.

- Momentarily setting S always leaves the latch in state $Q = 1$
- momentarily setting R always leaves it in state $Q = 0$.

The circuit *remembers* whichever was set last

Clocked SR Latch

To prevent the latch from changing state except at specific times, we add a clock input.



- Adding two AND gates gives a **clock** input, normally 0
- When `clock = 0`, both AND gates output 0 regardless of `S` and `R` - the latch holds its state
- When `clock = 1`, the AND gates have no effect, and the latch becomes sensitive to `S` and `R`

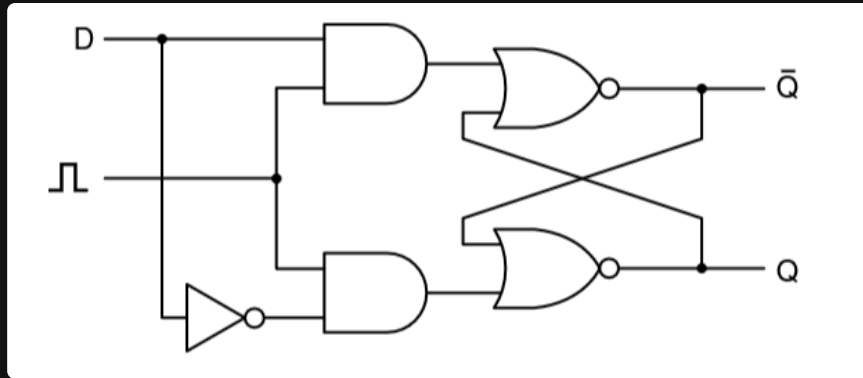
The terms **enable** and **strobe** are also used to mean "the clock input is 1."

Invalid State: $S = R = 1$

- The only consistent state for $S = R = 1$ is $Q = \bar{Q} = 0$
- But as soon as *both* inputs return to 0, the latch must jump to one of its two stable states
- If one input drops to 0 before the other, whichever stayed 1 longest **wins**
- If both return to 0 **simultaneously** (rare), the latch jumps to a state at **random**

Clocked D Latch

The SR latch's instability can be resolved by **preventing the bad case from ever occurring**.



- A single input, D , feeds the lower AND gate through an inverter, so the two AND gates never both see a 1
- When $D = 1$ and $\text{clock} = 1 \rightarrow \text{latch driven to } Q = 1$
- When $D = 0$ and $\text{clock} = 1 \rightarrow \text{latch driven to } Q = 0$

While the clock is 1, the current value of D is sampled and stored, giving a true 1-bit memory.

This circuit needs **11 transistors**.

Flip-Flops

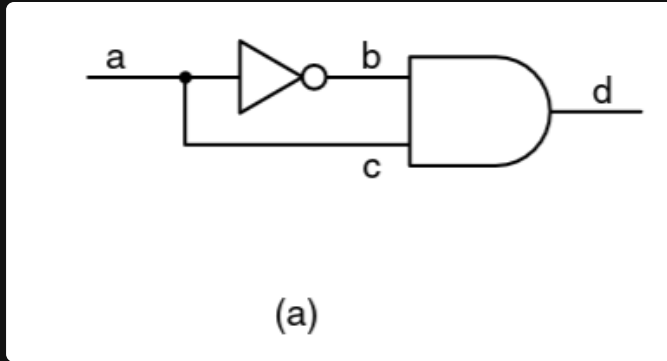
Often we need to sample a line at one *particular instant* rather than for as long as the clock is 1. This variant is a **flip-flop**.

- A flip-flop changes state on a clock **transition**, rising edge (0->1) or falling edge (1->0)
- A latch changes state while the clock is **level 1**

Flip-flop = edge triggered

Latch = level triggered.

Building a Flip-Flop



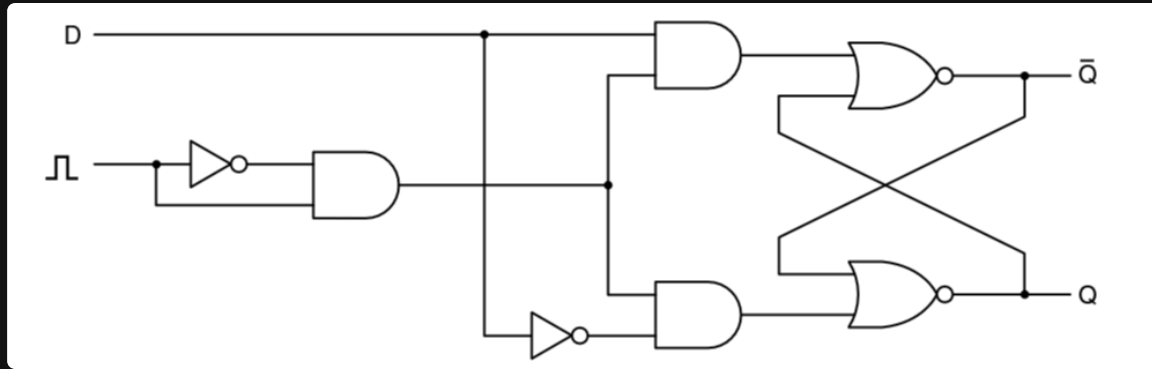
If we can generate a very short pulse on the rising edge of the clock, that pulse can drive a D latch directly.

An inverter has a small but nonzero **propagation delay**. ANDing the original signal with its delayed, inverted copy produces a short pulse of width Δ , equal to the inverter's gate delay (often 5 ns or less).

Physical propagation delay between measuring points (fractions of a nanosecond over microns of wire) is negligible next to the inverter's delay.

D Flip-Flop

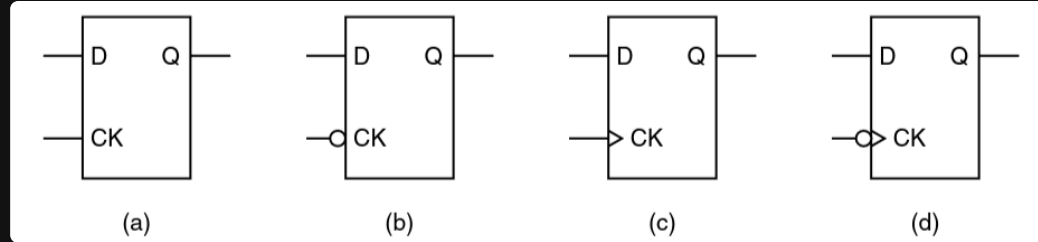
Feeding this short pulse into a D latch gives a complete **D flip-flop**.



- The flip-flop samples **D** at a fixed, tiny delay after the clock's rising edge
- In a memory with a 10 ns cycle time, a 1 ns sampling pulse is plenty short

Note: real memory circuits use more sophisticated, transistor-level designs than the gate-level explanations shown here.

Standard Symbols

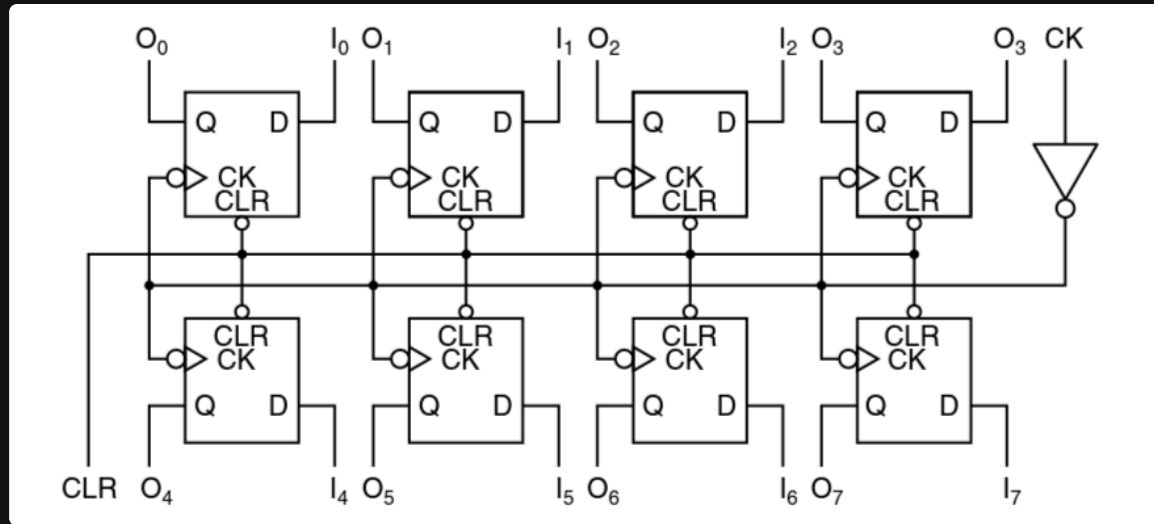


- **Latch**, clock normally 0, loads state while $CK = 1$
- **Latch**, clock normally 1, loads state while CK momentarily drops to 0
- **Flip-flop**, changes state on the clock's **rising** edge (pointy clock symbol)
- **Flip-flop**, changes state on the clock's **falling** edge

Many latches and flip-flops also expose $\bar{Q}$, plus extra **Set/Preset** (force $Q = 1$) and **Reset/Clear** (force $Q = 0$) inputs.

Registers

Flip-flops can be ganged together to hold data wider than 1 bit — a **register**.



- Eight flip-flops combine to form an **8-bit storage register**
- All eight clock lines are tied to a single **CK** signal, so every bit is accepted together on the clock transition
- All eight clear lines are tied to a single **CLR** signal, forcing every bit to 0 at once
- The input inverter on **CK** is really acting as an **amplifier**, since one signal may not have enough current to drive all eight flip-flop inputs directly

Building Larger Registers

Once an 8-bit register exists, it becomes a **building block** for wider registers.

- Two 16-bit registers, with their `CK` and `CLR` lines tied together, form a 32-bit register
- This same ganging technique scales to any register width a design needs